DOCENTE	**Fabio ABBA’**
UNITA’ FORMATIVA	**Competenze informatiche/digitali**
ARGOMENTO	**RESTful API**

Quattro chiacchiere tra macchine

Docente: Fabio Abbà

TC-Web
Technology Consulting

Intervento realizzato da
ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

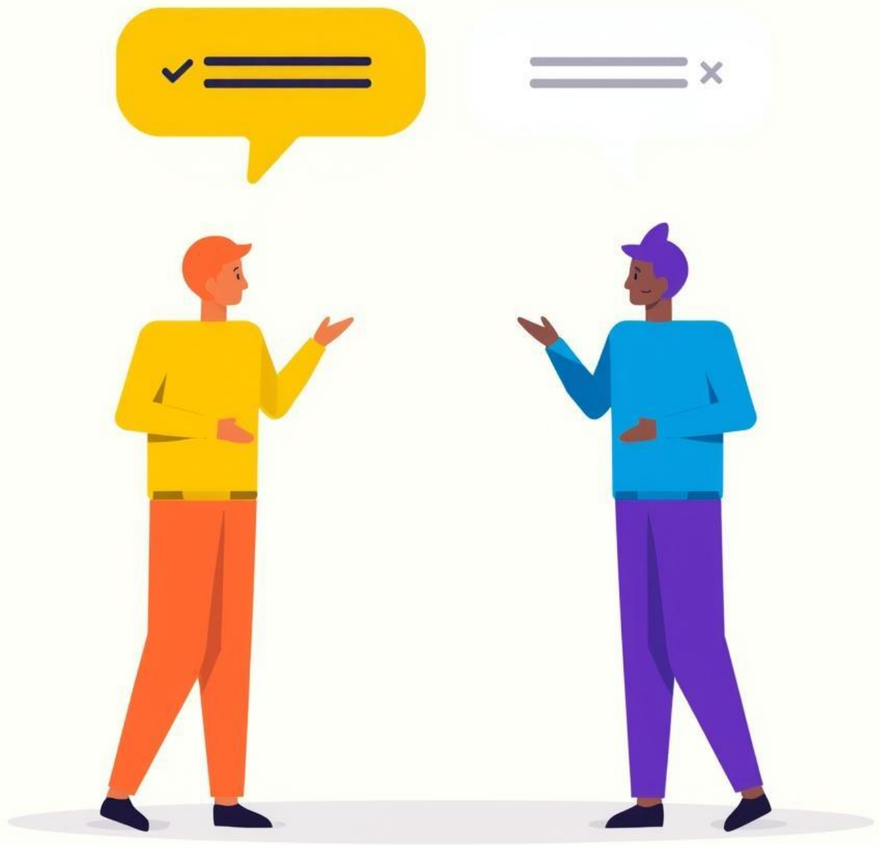


Comunicazione e interazione

Per comprendere la ragione di fondo di questo corso è utile fare un parallelo con ciò che facciamo tutti i giorni per interagire: parlare.

Perchè ci sia una buona comunicazione occorrono alcuni elementi:

- chi fa una richiesta (client)
- chi ascolta (server)
- un canale di trasmissione della voce/richiesta (web/http)
- una lingua comune (SOAP, REST, GraphQl, ecc.)



Distinzione tra Client e Server

Questo assicura che ci possa essere uno scambio efficace di informazioni tra due elementi.

Nel caso dei servizi web, è ormai prassi consolidata che ci sia una netta distinzione di ruoli tra client, che richiede e visualizza i dati, e server, che recupera ed invia i dati richiesti.
Questo ha anche il vantaggio di disaccoppiare le tecnologie utilizzate (p.es. è possibile modificare l'intera architettura di persistenza dati lato server in maniera completamente invisibile, o trasparente, per il client) e quindi permettere la comunicazione tra sistemi e linguaggi diversi.



Timeline delle origini

ARPANET

ARPANET, sviluppata negli anni '60, era la prima rete di computer, collegando istituzioni militari e università, ponendo le basi per la comunicazione digitale moderna.

TCP/IP

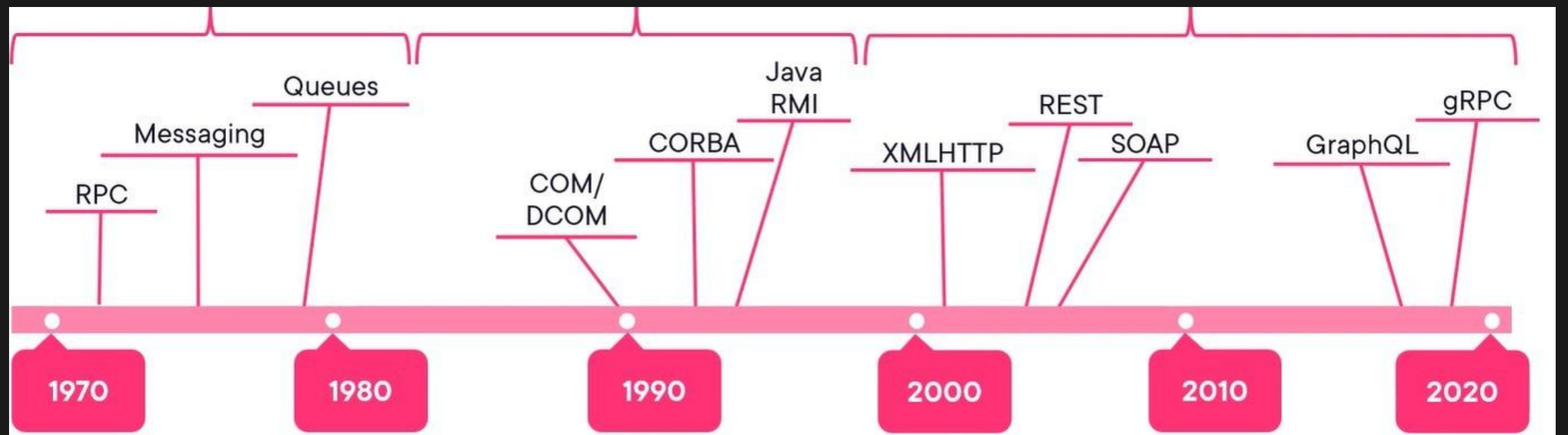
Il protocollo TCP/IP, adottato nel 1983, ha sostituito NCP, permettendo la comunicazione tra diversi sistemi operativi e migliorando l'affidabilità del trasferimento dati.

Protocollo HTTP

Il protocollo HTTP è stato introdotto negli anni '90, consentendo la trasmissione di contenuti web. Ha trasformato il modo in cui gli utenti interagiscono con le informazioni online.

Nascita del Web

Con la creazione di HTML e URL, Tim Berners-Lee ha dato vita al World Wide Web, rivoluzionando l'accesso alle informazioni e cambiando il panorama digitale per sempre.



HTML: Il linguaggio fondamentale del web

Hyper Text Markup Language, creato da Tim Berners-Lee, è un linguaggio di markup essenziale che definisce la struttura delle pagine web. Permette di posizionare e formattare contenuti visibili nei browser per l'utente finale.

```
<html>
  <head><title>Prima pagina web della storia</title></head>
  <body>
    <h1>World Wide Web</h1>
    <p>Benvenuti. Questo è un <a href="/info.html">link</a>.</p>
  </body>
</html>
```

Richieste HTTP

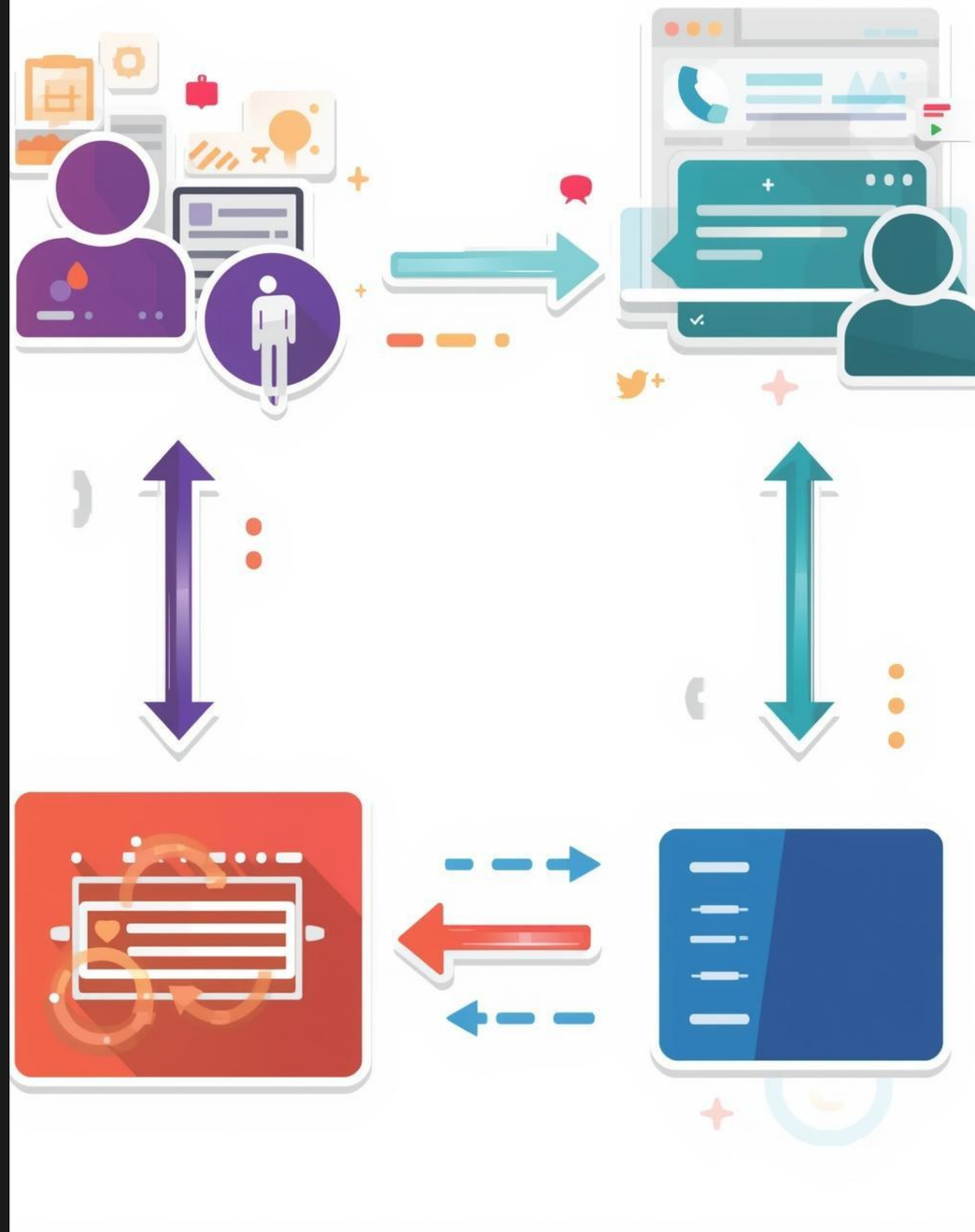
Esempio del primo HTTP, basata su due soli oggetti: request e response.

```
Richiesta del client:
  GET /index.html

Risposta del server:
<html>
  Contenuto della pagina...
</html>
```

Ma con l'evoluzione del web, è nata l'esigenza di fornire all'utente dei contenuti dinamici, ossia che si adattassero alle sue azioni o che potessero modificare il contesto applicativo: risultati di ricerca, form di contatto, pagine personalizzate.

Si giunge dapprima ad un'interattività lato server: Common Gateway Interface permetteva al server web di eseguire programmi esterni per generare HTML al volo.



Esempio di codice CGI in Perl



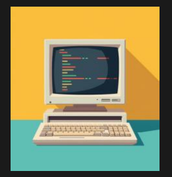
Come funzionava:

- Browser richiede /cgi-bin/hello.cgi
- Server esegue lo script Perl
- Script genera HTML e lo stampa su stdout
- Server invia l'output al browser

Esempio CGI in Perl:

```
#!/usr/bin/perl
print "Content-Type: text/html\n\n";
print "<html><body>";
print "<h1>Ciao, visitatore!</h1>";
print "<p>Sono le ", scalar(localtime), "</p>";
print "</body></html>";
```

Lato client: JavaScript e PHP



Discorso analogo con le Java Server Pages (.jsp) su cui qualcuno di vuoi probabilmente avrà ancora la (s)fortuna di lavorare...

Ma tutto questo era piuttosto grossolano e macchinoso, specie per le connessioni dell'epoca (le vecchie 56k dial up a pagamento... nel migliore dei casi).

Ben presto si scelse di far girare il codice per l'interattività lato client, direttamente nel browser per ridurre i tempi di risposta.

Così nel 1995 nacque PHP, da Java si staccò Javascript, nell'HTML si cominciarono a vedere tag <script> per inserire logiche applicative nelle pagine web.

```
<button onclick="alert('Ciao!')">Cliccami</button>

<script>
function validateForm() {
  var email = document.getElementById('email').value;
  if (!email.includes('@')) {
    alert('Email non valida!');
    return false;
  }
}
</script>
```

Evoluzione verso le Web Application



E via via, andando avanti, da un insieme di pagine interconnesse, poi di pagine dinamiche, si arriva al concetto di Web Application, e anzi, di Single Page Web Application: interi applicativi che girano nel browser dell'utente in costante comunicazione con il server.

Roy Fielding, uno dei principali autori della specifica HTTP/1.1, ha definito **REST** *(Representational State Transfer)* nella sua dissertazione di dottorato del 2000 presso l'Università della California. Fielding, avendo lavorato direttamente su HTTP, aveva osservato cosa funzionava e cosa no nel protocollo che alimentava il web. La sua intuizione fu: perché inventare nuovi protocolli quando HTTP stesso, ben utilizzato, poteva essere la soluzione?

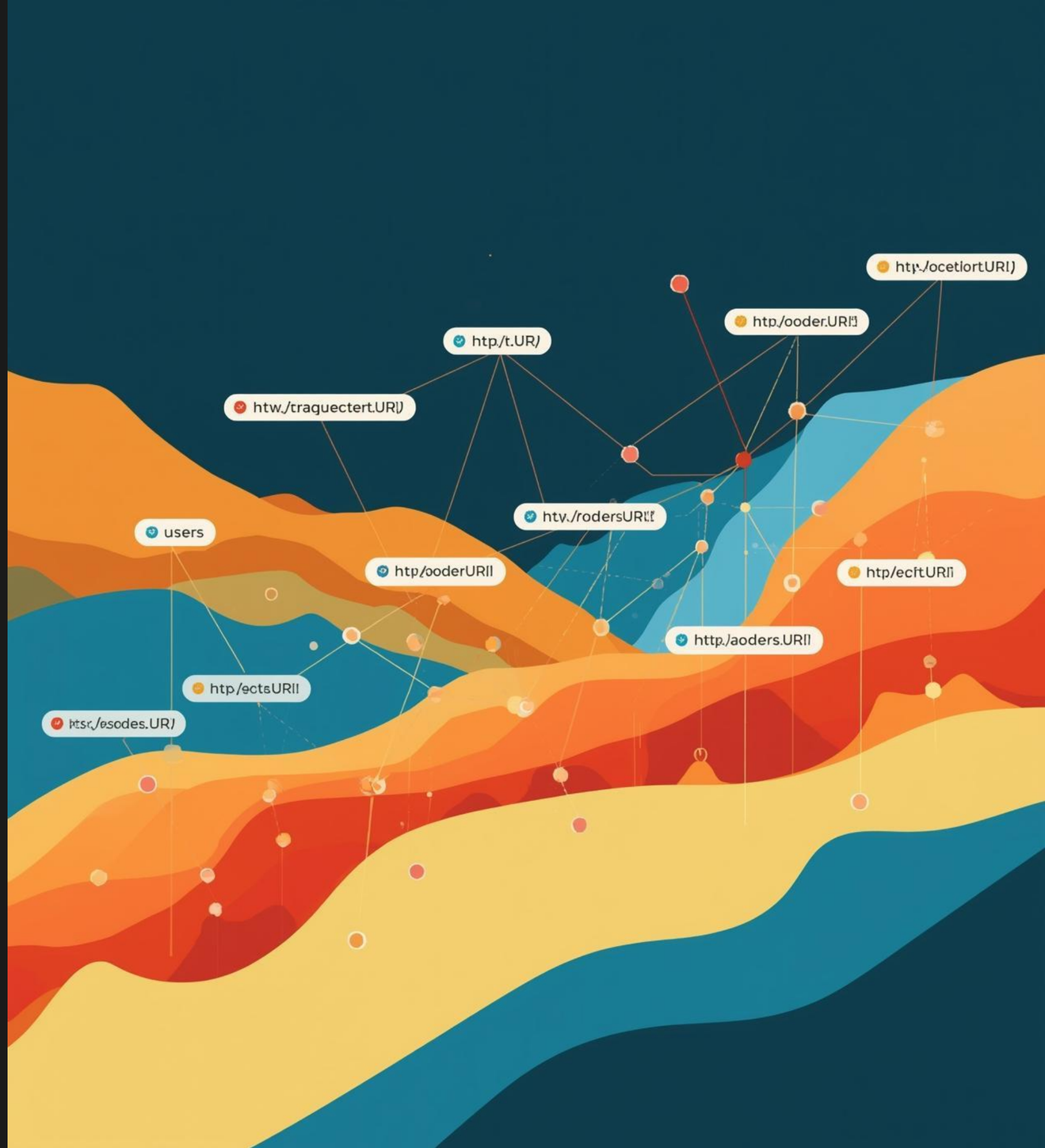
Quindi non inventò (quasi) nulla di nuovo, ma identificò e formalizzò i vincoli architetturali che rendono il web scalabile.

Le risorse e i loro URI

Fielding ha introdotto il concetto che tutto nel web può essere modellato come risorsa identificata da un URI. Non "servizi" o "procedure remote", ma ***RISORSE*** (qualsiasi elemento oggetto di elaborazione)

Un utente è una risorsa: */users/123*
I suoi ordini sono una risorsa: */users/123/orders*
Un ordine specifico: */orders/456*

Questo cambio di prospettiva ha reso le API più intuitive e prevedibili.

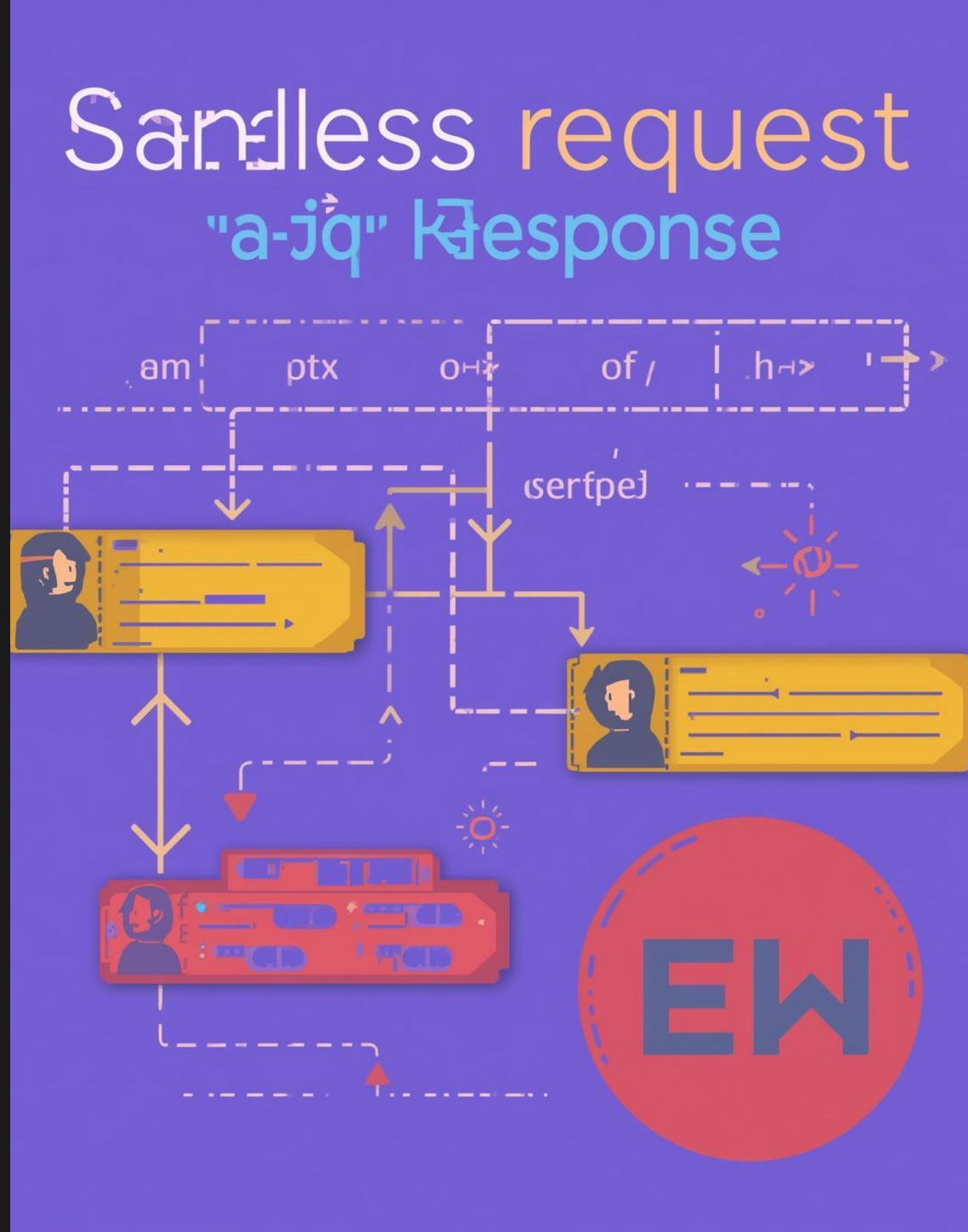


Stateless Design

Stateless: ogni richiesta deve contenere tutte le informazioni necessarie. Il server non mantiene sessioni tra richieste.

Questo implica che il client ha la responsabilità di mantenere lo stato della sessione e le informazioni di autenticazione.

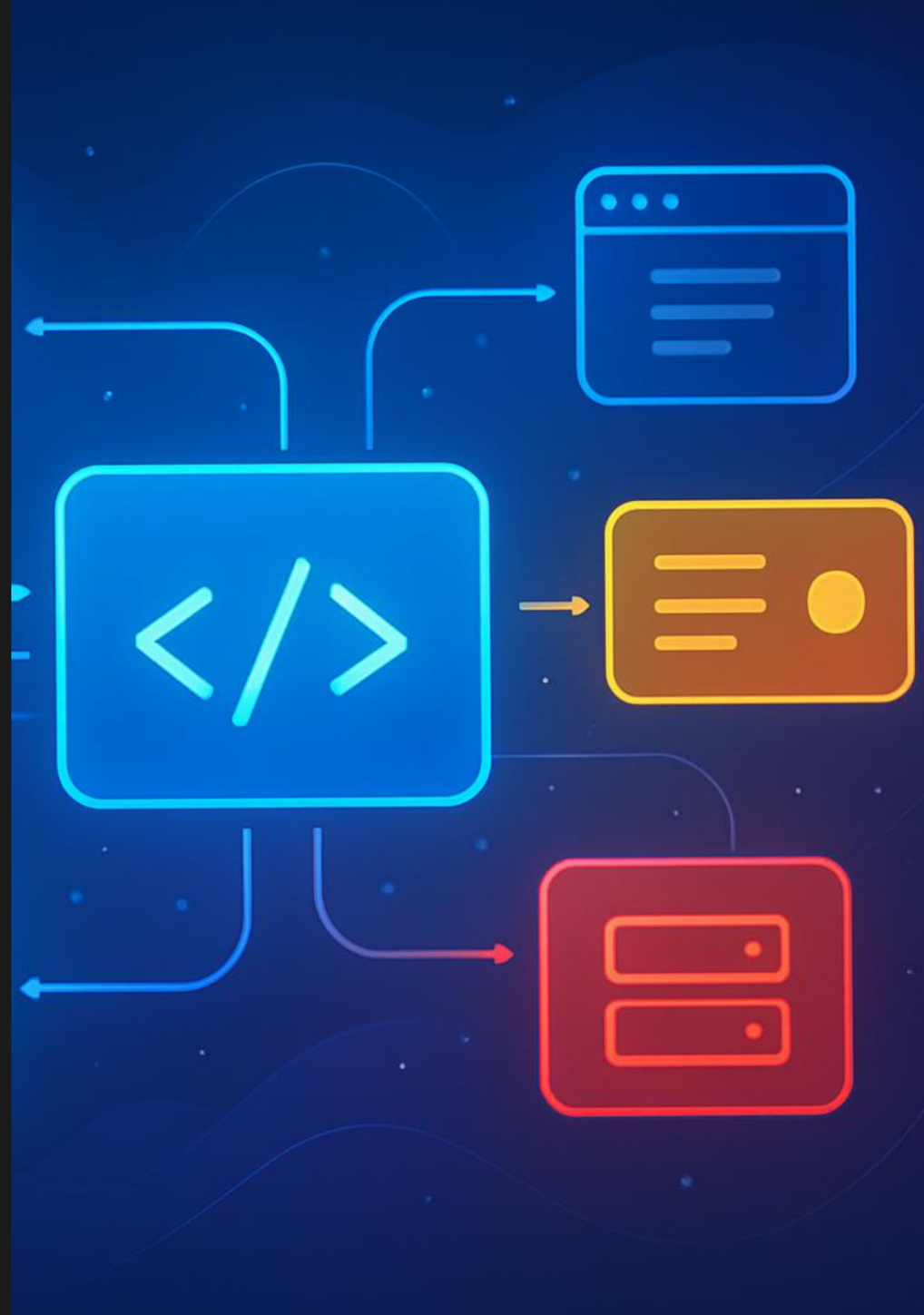
Perché? Scalabilità. Se il server non deve ricordare lo stato delle conversazioni, rimangono più risorse libere e le richieste sono gestibili anche da server diversi.



HATEOAS

```
{
  "order_id": "456",
  "status": "pending",
  "_links": {
    "self": { "href": "/orders/456" },
    "cancel": { "href": "/orders/456/cancel" },
    "pay": { "href": "/orders/456/payment" }
  }
}
```

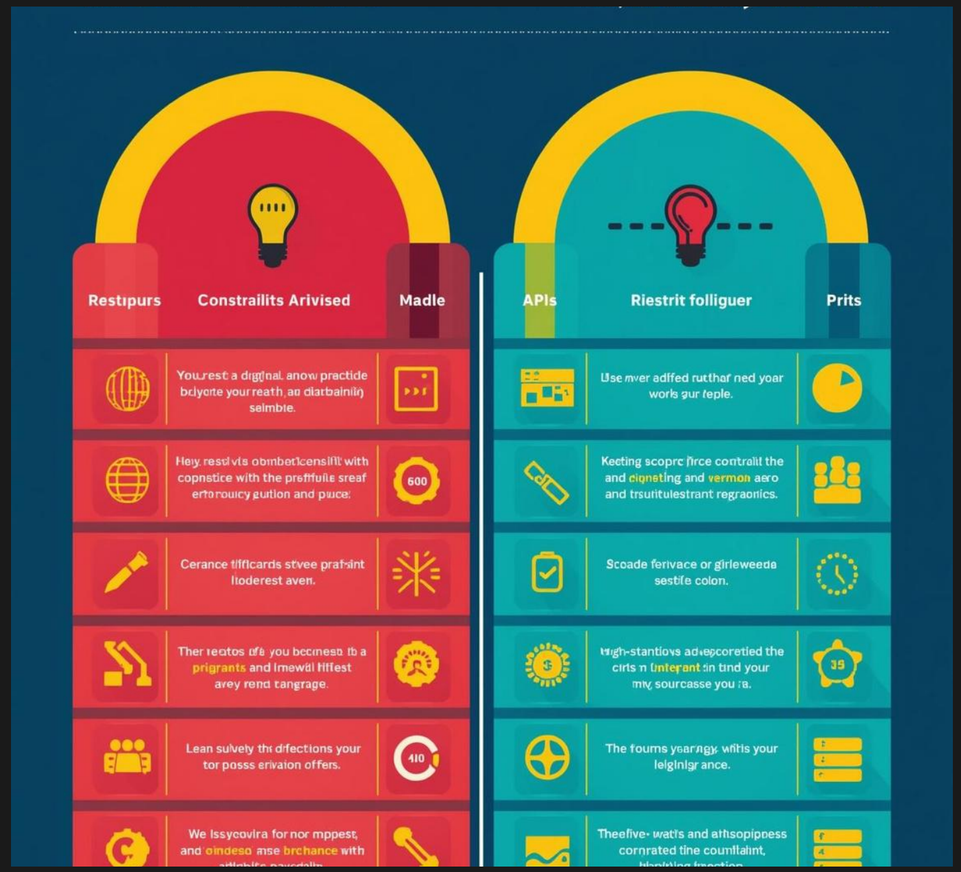
HATEOAS (Hypermedia As The Engine Of Application State) è il vincolo più avanzato (e meno implementato) di REST. L'idea è che la risposta dovrebbe contenere link alle azioni successive possibili.



I 6 comandamenti REST

Per garantire una piena interoperabilità tra sistemi, le API sviluppate con l'architettura REST seguono questi 6 principi:

- ***Interfaccia uniforme*** (risorse identificate da URI, metodi HTTP, rappresentazione oggetti JSON/XML/…, eventualmente HATEOAS)
- ***Disaccoppiamento client/server*** (portabilità interfaccia)
- ***Comunicazioni Stateless*** (richieste autoconsistenti)
- ***Cacheability*** (possibilità di memorizzare le risposte in cache)
- ***Architettura a strati*** (possibilità di livelli intermedi, proxy, gateway…)
- ***Codice on demand*** (OPZIONALE; il server può inviare applet o script per estendere le funzionalità dell'applicazione)



Pratica con PokeAPI



Ora mettiamo in pratica quanto appreso esplorando la **PokeAPI**. Utilizzeremo questa API RESTful per effettuare richieste e ricevere dati sui Pokémon.

https://pokeapi.co/

Verbi HTTP: le azioni fondamentali di REST

Ok, ora aggiungiamo ancora qualche informazione.

I **verbi HTTP** (o metodi HTTP) definiscono il tipo di operazione che vogliamo eseguire su una risorsa. Sono il "vocabolario" con cui comunichiamo con un'API REST.

Il protocollo ne prevede 5 principali:

- GET
- POST
- PUT
- PATCH
- DELETE



I 5 verbi principali

GET - Recupera una risorsa o collezione

- Esempio: `GET /users/123` → ottiene i dati dell'utente 123
- È "safe" (non modifica nulla) e "idempotente" (chiamarlo più volte dà lo stesso risultato)

POST - Crea una nuova risorsa

- Esempio: `POST /users` con dati nel body → crea un nuovo utente
- Non è idempotente: ogni chiamata crea una nuova risorsa

PUT - Sostituisce completamente una risorsa esistente

- Esempio: `PUT /users/123` → aggiorna tutti i campi dell'utente 123
- È idempotente: ripetere la stessa richiesta non cambia il risultato



I 5 verbi principali

PATCH - Aggiorna parzialmente una risorsa

- Esempio: `PATCH /users/123` con `{"email": "nuovo@email.com"}` → modifica solo l'email
- Invia solo i campi da cambiare, non l'intera risorsa

DELETE - Rimuove una risorsa

- Esempio: `DELETE /users/123` → cancella l'utente 123
- È idempotente: dopo la prima cancellazione, la risorsa non esiste più



Perchè sono importanti

I verbi HTTP rendono l'API prevedibile e intuitiva.

Invece di avere URL come `/getUserById` o `/deleteUser`, usiamo sempre lo stesso pattern: il verbo indica l'azione, l'URL indica la risorsa.
Questo è il cuore della filosofia REST.

Riprendendo l’analogia della comunicazione umana, questo è il contenuto della comunicazione, ciò che viene effettivamente richiesto.



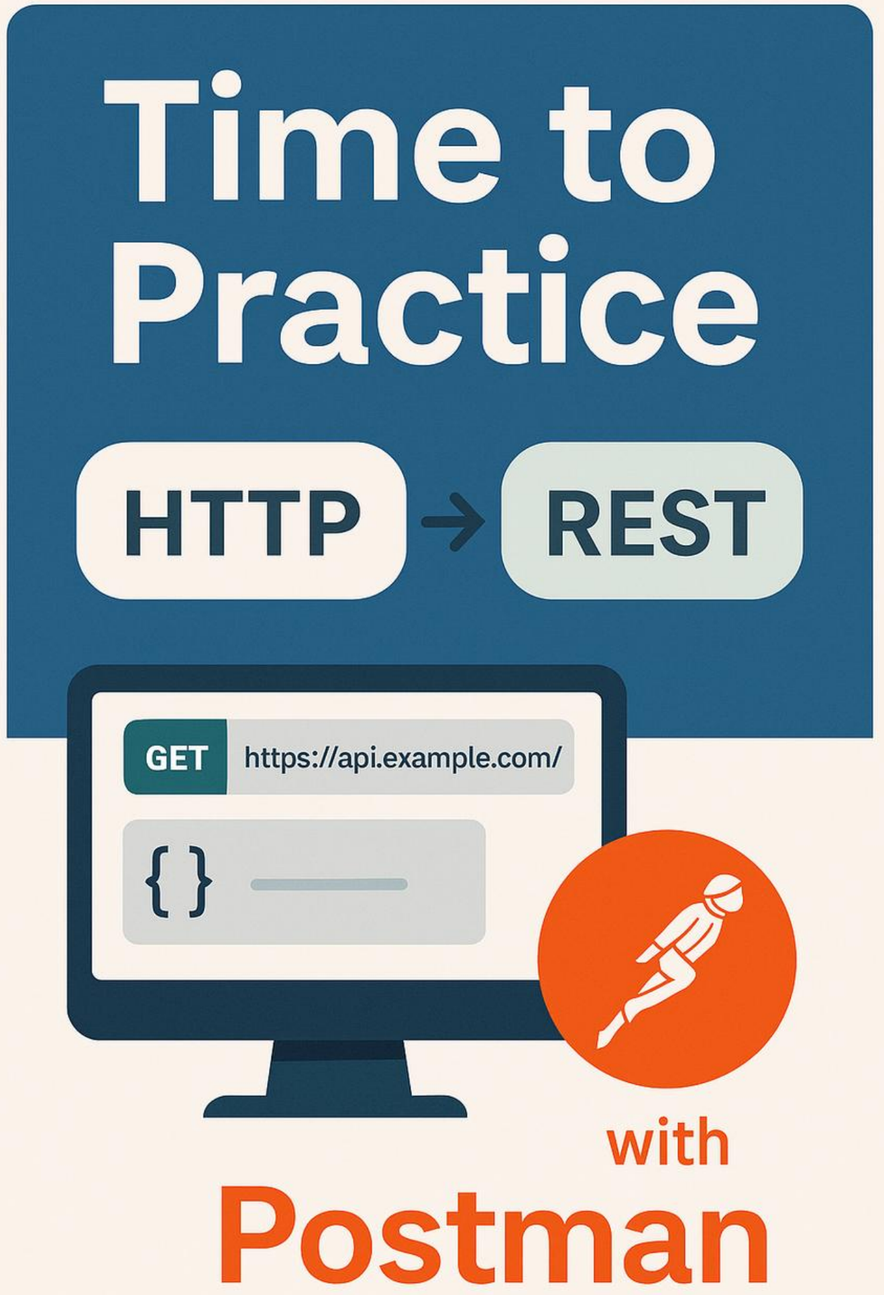
E’ ora di testare!

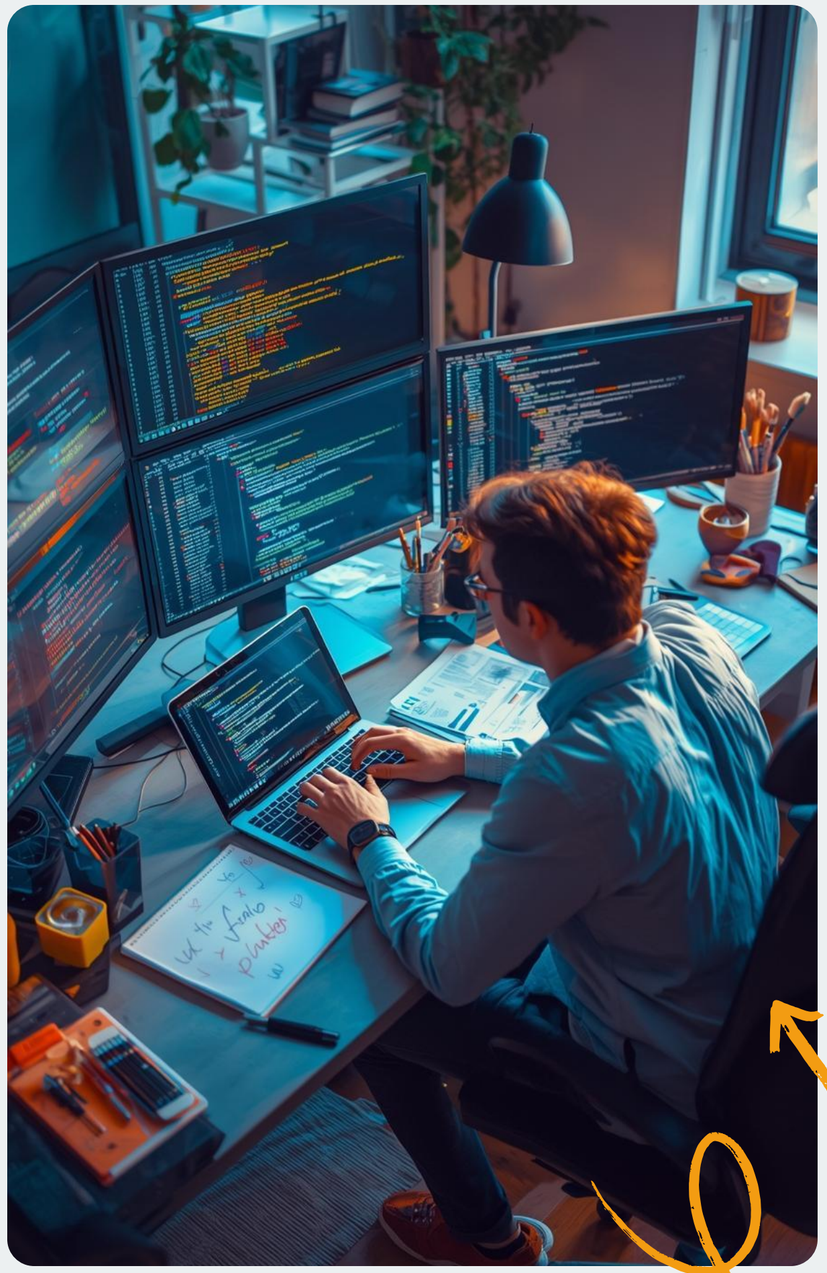
Ok, per il momento basta con la teoria, è ora di mettere le mani in pasta.

Lo faremo utilizzando un tool utilizzo molto comune:

- Postman (https://www.postman.com)

E alcune API pubbliche:
https://swapi.dev
https://jsonplaceholder.typicode.com





COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

Grazie per l'attenzione!

Docente: ***Fabio Abbà*** *fabio.abba@tc-web.it*

Credits: Franco Grivet

franco.grivet@tc-web.it

Intervento realizzato da

ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

TC-Web Technology Consulting

Via Giovanni Schiaparelli 14,
10148 – Torino (TO)
E-mail: info@tc-web.it
Tel: 011 0466685